Deep Reinforcement Learning

Idlir Shkurti

May 24, 2015

Contents

1 Introduction **3**

1.1 Machine Learning Problems 3

1.2 Background 4

1.2.1 Bayesian Decision Theory 5

1.2.2 Markov Decision Process 6

1.2.3 Environment Interaction 6

1.2.4 Value Functions 7

1.2.5 Optimal Decisions 8

2 Deep Reinforcement learning **10**

2.1 Neural Networks 10

2.2 Deep Q-Learning 11

3 Experiment **13**

3.1 Playing A Game Of 'Catch' 13

3.1.1 Performance Diagnostics 15

3.1.2 Diminishing Epsilon Performance 16

3.1.3 Varying Layer Size Performance 17

4 Conclusions And Further Work **21**

5 Appendix **23**

List of Figures

1.1 The agent-environment interaction in reinforcement learning. Figure taken from [Sutton and Barto, 1998] 7

2.1 A graphical representation of a neural network with one hidden layer. . . . 10

2.2 **Left**: A conventional Q-network. **Right**: Deep Q-network introduced in DeepMind paper. 12

3.1 Using *keras* deep learning library to play a game of 'catch' 13

3.2 Model Training Loss And Win Count 16

3.3 Q-value Diagnostics 17

3.4 Training vs Testing Accumulated Reward 18

3.5 Training set performance for different values of ϵ 19

3.6 Training performance for different hidden layer sizes 20

Chapter 1

Introduction

1.1 Machine Learning Problems

Machine learning is, and has been for the past couple of decades, one of the most exciting research areas of computer science and engineering. The idea that a computer software could have the ability to learn and improve performance without being explicitly programmed to do, does not only come with a academic research interest but also vast applications within a wide range of industries. Machine learning problems are usually classified into the following 3 different categories:

1. Unsupervised learning
2. Supervised learning
   (a) Regression
   (b) Classification
3. Reinforcement learning

These problems arise due to different availability of feedback.

Unsupervised learning methods assume no feedback from the system. This means that no labels are given to learning algorithm. The system must detect the important features and hidden patters of the data, solely based on the input.

Supervised learning techniques are used when the data are given not only the inputs but also the desired outputs. In other words, given 'labelled' observations $x_1, x_2, ..., x_n$ and their corresponding response variables $y_1, ..., y_n$, we observe $\{(x_i, y_i)\}_{i=1}^n$. The goal of supervised learning is to correctly predict outcomes y_i for new inputs x_i. There are typically two types of supervised learning problems. *Classification* problems arise when we deal with discrete outcomes, e.g. $Y \in \{-1, 1\} \quad or \quad \{1, ..., K\}$. *Regression* problems arise when the outcome is numerical and continuous, $Y \in R$.

Reinforcement learning occurs when computer program interacts with a dynamic environment in which it must perform a certain goal. Reinforcement learning is concerned with how an agent can perform actions to receive feedback from the environment. The feedback

is provided in terms of reward or punishment from the dynamic environment. Reinforcement learning differs from supervised learning. In supervised learning we build machine learning models based on a training set where all the outcomes are known. It is often very difficult with a dynamic environment to be able to decide whether a certain behaviour of the agent is correct, hence reinforcement learning, an agent must be able to learn from its own experience.

1.2 Background

Ever since the first computers were created, the ultimate aim of artificial intelligence has been to build machines which can perform tasks better than humans. From Charles Babbage's mechanical computer to IBM's Deep Blue being the first computer system to beat a reigning chess world champion, computer scientists have proved that machines can be specifically tailored to perform certain tasks more accurately and efficiently than humans. However, a truly intelligent software would be able to learn from its mistakes and improve performance towards a certain goal, without actually being programmed to do so. Deep Blue was a computer software built specifically to play chess, Turing's original Enigma decrypting machine was built specifically to aid mathematicians in code decryption. Could a universal algorithm be used in different objective scenarios in order to receive feedback and improve its own performance?

Over the past decade, companies in various fields such as fintech and healthcare have invested into reinforcement learning research as a potential self-learning data analytic tool. One of the most exciting companies currently working on deep reinforcement algorithms is Deepmind. Similarly to many artificial intelligence research groups, Deepmind believe that these algorithms can be implemented in order to understand and control the behaviour of complex multi-agent systems such as the economy, traffic, and environmental challenges. So far the algorithms which are used from Deepmind or other projects such as Google Brain have been efficient when used towards reaching different objectives. The main issue with this ideology has been that building one algorithm is never going to emulate the human brain. The human brain is complicated and different parts of the brain work on specific tasks. An interesting experiment from [Roe et al., 1992] gave birth to the so called 'One Algorithm' hypothesis. The "input" for vision within a group of ferrets was plugged into auditory part of brain in order to observe whether the auditory cortex of the ferrets could learn to "see". These experiments were successful and gave evidence to suggest that the human brain uses essentially the same algorithm to understand many different input modalities.

One of the challenges that arise in reinforcement learning and not in other kinds of learning is the trade-off between exploration and exploitation [Sutton and Barto, 1998]. In order to obtain a higher reward and reach its objective, an agent must prefer actions and decisions which it has made in the past. Nevertheless, exploration should be an essential feature of these agents. Exploration could help these agents discover more efficient methods of performing certain tasks.

n-Armed Bandit Problem

Consider the following: an agent at a certain point in time is faced with n possible actions to take. Each action comes with a numerical reward from a stationary probability distribution. The agent must take a certain pre-specified number of these decisions and its objective is to maximise the final reward. The analogy of an 'n-armed bandit' comes from slot machines - one armed bandits, except the machine has n levers. Multi-armed bandit problems are the most basic examples of sequential decision problems with an exploration-exploitation trade-off. This is the balance between staying with the option that gave highest pay-offs in the past and exploring new options that might give higher pay-offs in the future [Bubeck et al., 2012]. Each time the agent performs the task the rewards from each action are rewarded and each action will have an expected reward associated with it, called the *value* of that action. Since the rewards from each action come from a stationary probability distribution, there is always some uncertainty in the reward estimates of the agent. If the agent continues to maintain and upgrade action values, there will be at any time at least one action whose estimated value is greatest which we call the *greedy action*. Choosing a greedy action means the agent is exploiting the current knowledge of the action values. Choosing any other action means the agent is exploring. Exploring is important because it improves the value estimates of the non-greedy actions.

1.2.1 Bayesian Decision Theory

A discrete-time stochastic process $X_1, X_2, X_3, ...$ is said to be a Markov process [Dynkin] if the probability of moving to each state depends only on the previous state:

$$P(X_n|X_{n-1}, ..., X_2, X_1) = P(X_n|X_{n-1}) \tag{1.1}$$

In Bayesian Decision Theory we are concerned with making optimal sequential decisions which maximise the reward (or equivalently minimise the loss). We must assume the following:

- We have a choice of taking k possible **actions** $a_1, ..., a_k$;
- The agent can be in m possible **states** $s_1, ..., s_m$;
- If we take action a_i and the agent is in state s_j, the agent receives a **loss** l_{ij};
- Given all the observed data D and prior background knowledge B, our beliefs about the state of the world are summarized by $p(s|D, B)$.

The optimal function is one that is expected to minimise the loss (or equivalently maximise the reward)

$$a^* = argmin_{a_i} = \sum_{j=1}^{m} l_{ij} p(s_j|D, B) \tag{1.2}$$

We consider problems where the action chosen by an agent influences its state. We use the previous assumptions to solve Markov sequential decision process.

1.2.2 Markov Decision Process

A reinforcement learning task that satisfies the Markov property is called a Markov decision process (MDP) [Sutton and Barto, 1998]. In this paper we will only consider finite MDPs[1], where the state and action spaces are finite.

An MDP is a tuple (S, A, p, r) where:

- S is a set of states;
- A is a set of actions;
- $p(z|x, a)$ is a transition distribution defining the probability of moving to state $z \in S$ from state $x \in S$ on action $a \in A$;
- $r(x)$ is a reward function.

Given any state and action, x and α, a particular MDP is defined by its *transition probabilities* from state x to state z

$$P^a_{z,x} = P(s_{t+1} = z|s_t = x, a_t = \alpha) \tag{1.3}$$

and *expected reward*:

$$R^a_{z,x} = E(r_{t+1}|s_{t+1} = z, s_t = x, a_t = \alpha) \tag{1.4}$$

1.2.3 Environment Interaction

Every machine learning model consists of a learner which we call the *agent* and the thing it interacts with which compromises everything outside the agent called the *environment*. The agent and the environment interact at every discrete time step[2] $t = 1, 2, 3, ...$ At each time t, the agent receives from the environment some representation of the current state s_t and based on this chooses an action a_t which brings the agent to state s_{t+1}. On the next time step $t + 1$ the agent receives a reward r_{t+1}. This agent-environment interaction algorithm is shown in figure 1.1.

At each time point t, there exists a mapping from states to probabilities. This assigns probabilities to all possible actions which can be taken from time t. We call this probability assignment the agent's *policy* and denote it by $\pi_t(a, s)$. This is the probability that at time t, the agent takes action a from state s. In reinforcement learning, the agent usually changes the policy π_t after interacting with the environment and gaining experience from the environment's feedback.

[1] Finite MDPs are the most widely used MDPs in Reinforcement Learning. Fern (2004) present interesting metrics for measuring the similarity of states in a finite Markov decision process.

[2] Ideas can be extended to the continuous-time case. However we only consider discrete time here to keep things simple. [Bertsekas, 1996] give examples of neuro-dynamic programming for continuous time cases.

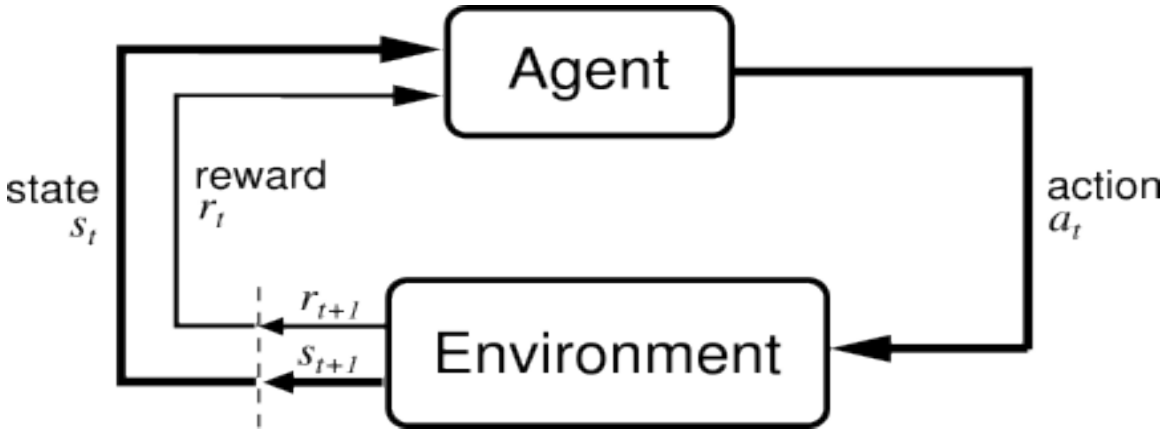


Figure 1.1: The agent-environment interaction in reinforcement learning. Figure taken from [Sutton and Barto, 1998]

In a task of T time steps, the role of the agent at any time t is to maximise the expected return. The return in the simplest case can be written as the sum of the rewards:

$$R_t = r_{t+1} + r_{t+2} + r_{t+3} + ... + r_T \tag{1.5}$$

However a reward r received k steps in the future is usually not worth the same as it would be if received now. This brings the need for the concept of **discouting**. We define the *discounted return* as:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + ... = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \tag{1.6}$$

where γ is a parameter $0 \leq \gamma \leq 1$ called the discount rate. Hence the agent usually maximises the discounted rate, taking γ into consideration.

1.2.4 Value Functions

Almost all reinforcement learning problems are based on *value functions*. We define the value of a state s under a policy π, denoted $V^\pi(s)$, as the expected return when starting in s and following policy π thereafter:

$$V^\pi(s) = E_\pi(R_t|s_t = s) = E_\pi\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}|s_t = s\} \tag{1.7}$$

Where E_π denotes the expected value given the agent follows policy π. Similarly we define the *action-value* function $Q^\pi(s, a)$ as the expected return of the agent, starting at state s, taking action a and then following policy π.

$$Q^\pi(s) = E_\pi(R_t|s_t = s, a_t = a) = E_\pi\{\sum_{k=0}^{\infty} \gamma^k r_{t+k+1}|s_t = s, a_t = a\} \tag{1.8}$$

The value function and the action-value function can be estimated by **Monte Carlo** non-parametric methods from experience. As the agent completes the task several times following policy π, the actual rewards following each state and taking each action are taken. The averages of these rewards as the number of completed tasks tends to infinity converge to the state's value $V^{\pi}(s)$ and action value $Q^{\pi}(s,a)$.

1.2.5 Optimal Decisions

At any state $s \in S$ where S is the set of all possible states, there is a policy π which is better than or equal to all other policies. A policy π is said to be better than or equal to a policy π' if $V^{\pi}(s) \geq V^{\pi'}(s)$ for all states s. The policy that is better than or equal to all other policies is called the *optimal policy* and is usually denoted by π^*. We define the *optimal state-value function* $V^*(s)$ and *optimal action-value function* $Q^*(s,a)$ as:

$$V^*(s) = max_{\pi} V^{\pi}(s) \tag{1.9}$$

$$Q^*(s,a) = max_{\pi} Q^{\pi}(s,a) \tag{1.10}$$

for all $s \in S$ and $a \in A(s)$. It is worth noting that the exploitation-exploration problem arises here. The agent would have to 'decide' whether to follow optimal policy or to explore the other policies and later use the experience to improve the Monte-Carlo approximations of the value functions.

The optimal action-value function obeys the *Bellman equation* which is based on the following: if the optional value $Q^*(s',a')$ is known for any state s' at any time point for all actions a', then the optimal strategy is to take action a' maximising the expected value of $r + \gamma Q^*(s',a')$,

$$Q^*(s,a) = E_{s' \sim \epsilon}(r + \gamma max_{a'} Q^*(s',a')|s,a) \tag{1.11}$$

where ϵ is the environment the agent is interacting with. Reinforcement learning algorithms attempt to estimate the action-value function using the Bellman equation as an iterative update:

$$Q_{i+1}(s,a) = E_{s' \sim \epsilon}(r + \gamma max_{a'} Q_i(s',a')|s,a) \tag{1.12}$$

$$Q_i \rightarrow Q^* \quad as \quad i \rightarrow \infty \tag{1.13}$$

In section 1.2 we defined a greedy action $a*$. An agent which only chooses greedy actions does not explore the remaining actions and might miss out on a higher final return. A simple method for ensuring adequate exploration is by assigning some level of stochasticity to the agent's decisions. The agent can behave greedily most of the time but with a probability ϵ it will explore the other actions at random. Methods which use this are called **ϵ-greedy methods**. These methods rise a number of concerns such as which value of ϵ to use and that the greedy methods actually perform significantly better at first in terms of accumulated reward. However these near-greedy methods outperform the greedy methods in the long run as the greedy agents tend to get stuck performing suboptimal actions.

A widely used model-free algorithm used for estimating the action-value function is the *Q-learing* algorithm [Watkins and Dayan, 1992]. This algorithm also takes care of the

exploration-exploitation problem since it ensured adequate exploration of the non-greedy actions. Algorithm 1 shows the gist of the Q-learning algorithm where α is the learning rate that controls how much of the difference between previous Q-value and newly proposed Q-value is taken into account. When $\alpha = 1$ the update is exactly the same as the Bellman equation.

Algorithm 1 Q-learning algorithm using Bellman equations

```
Initialise Q(num_states, num_actions) arbitrarily.
Observe initial state s
repeat
Select and carry out action a following an ϵ-greedy policy
Observe reward r and new state s'
Q(s, a) = Q(s, a) + α[r + γ max_a' Q(s', a') − Q(s, a)]
Let s = s'
until terminated
```

Chapter 2

Deep Reinforcement learning

2.1 Neural Networks

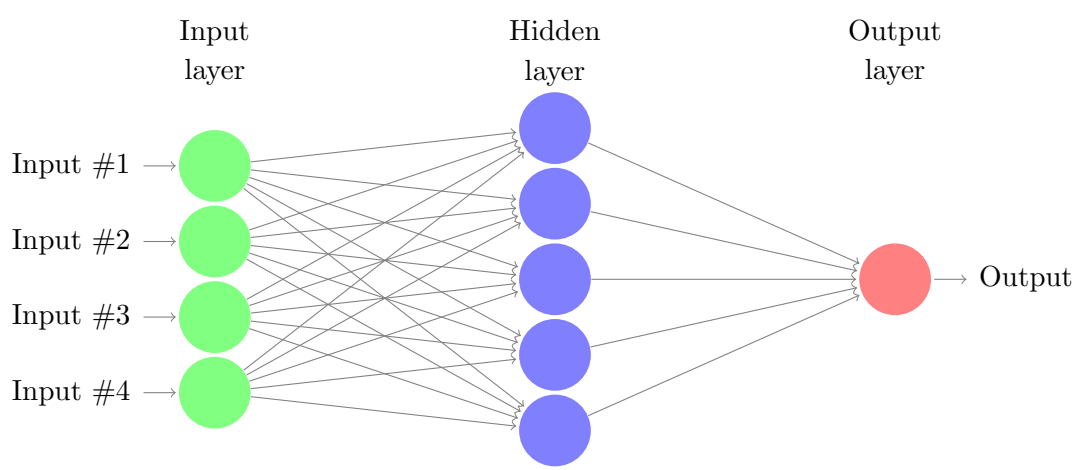


Figure 2.1: A graphical representation of a neural network with one hidden layer.

Neural networks are discriminative learning methods used in machine learning. Neural networks are widely used in supervised learning problems for classification or regression. Figure 2.1 gives a graphical representation of a neural network with one input layer, one hidden and one output layer. The inspiration for these networks comes from biological neurons which receive signals from other neurons via dendrites and send processed signals via axons. The neurons in each layer of a neural network are the basic computational elements. in the hidden layers they are trained to give appropriate weights to each input which comes from the previous layer. In multiple hidden layer problems these weights are determined through an iterative process called the backpropagation algorithm which uses stochastic gradient descent [Hecht-Nielsen et al., 1988] [Hinton and Salakhutdinov, 2006]. The neuron in the output layer uses a activation function s in order to make the final predictions. A common non-linear activation function used for classification problems is the *inverse logit* function (logistic function) $s(a) = \frac{1}{1+e^{-a}}$.

Since stochastic gradient descent can be very computationally expensive, adaptations to the Q-learning algorithms are typically used in order to estimate the action-value function.

Action-value iteration algorithms are useful because as $i \to \infty$, $Q_i \to Q^*$. This is typically impractical since the action-value function is approximated separately for each iteration. Using an approximate function $Q(s, a; \theta) \approx Q^*(s, a)$ is more efficient. θ are the weights of a neural network function approximator (also called a Q-network). A Q-network, just like any other neural network, is trained by minimising a certain loss function. The loss function here $L_i(\theta_i)$ changes with every iteration i:

$$L_i(\theta_i) = E_{s,a}[(y_i - Q(s, a; \theta_i))^2] \tag{2.1}$$

where $y_i = E_{s'}(r + \gamma max_{a'} Q(s', a'; \theta_{i-1})$ is the target for iteration i. The parameters from the previous iteration θ_{i-1} are held fixed when optimising the loss function $L_i(\theta_i)$.

2.2 Deep Q-Learning

A technique known as *experience replay* [Lin, 1993] is used to store the agent's experience at each time step $e_t = (a_t, s_t, r_t, s_{t+1})$ in dataset $D = e_1, ..., e_N$. Deepmind introduced an algorithm which uses this dataset of experiences to perform Q-learning updates. The agent then selects and executes an action a_{i+1} according to an ϵ-greedy policy. The full algorithm presented by Deepmind in [Mnih et al., 2013], which Deepmind call *deep Q-learning* is presented below.

Algorithm 2 Deep Q-learning with Experience Replay

Initialize replay memory D to capacity N
Initialize action-value function Q with random weights
for $episode = 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    Otherwise select $a_t = max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and pre-process $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_t + 1)$ in D
    Sample random minibatch of transitions $(\phi_t, a_t, r_t, \phi_t + 1)$ from D
    $set \quad y_j = \begin{cases} r_j & for \quad terminal \quad \phi_{j+1} \\ r_j + \gamma max_{a'} Q(\phi_{j+1}, a'; \theta) & for \quad non \quad terminal \quad \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$
  end for
end for

This algorithm used by Deepmind [Mnih et al., 2013] is much more efficient than previous deep Q-learning algorithms. Ultimately we want a Q-network that is also a good Q-value predictor for unknown states. We could represent our Q-function with a neural network, that takes the state and action as input and outputs the corresponding Q-value. Alternatively, Deepmind proposed taking only the states as input and output the Q-value for each possible

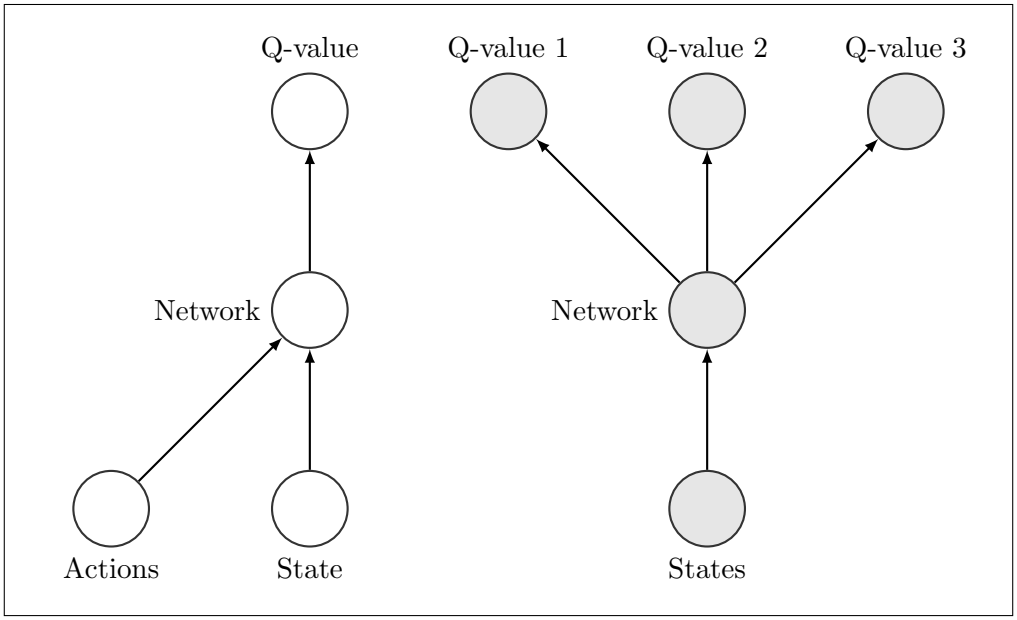


Figure 2.2: **Left**: A conventional Q-network. **Right**: Deep Q-network introduced in DeepMind paper.

action in those particular states. This approach has the advantage, that if we want to perform a Q-value update or pick the action with the highest Q-value, we only have to do one forward pass through the network and have all Q-values for all actions available immediately. A graphical representation of the difference between the conventional deep Q-network and the Q-network used by Deepmind in algorithm 2 is shown in figure 2.2 where disparate Q-values are estimated at each iteration[1].

The convolutional neural network used by Deepmind is the following:

Layer	**Input**	**Activation**	**Output**
conv1	84x84x4	ReLU	20x20x32
conv2	20x20x32	ReLU	9x9x64
conv3	9x9x64	ReLU	7x7x64
fc4	7x7x64	ReLU	512
fc5	512	Linear	18

Table 2.1: The network architecture used by Deepmind

where the first 3 layers are classical three-dimensional concolutional layers, the last 2 are fully connected layers and ReLU activation stands for Rectified Linear Unit. The inputs to the system are four 84x84 game screens and the outputs are the Q-values for each possible action, making this a regression task rather than classification. The Deepmind example had 18 possible actions since it learned how to play Atari games and hence 18 Q-value outputs. when training the network, random mini-batches are used instead of the most recent transition. This mitigates the problem of the network getting stuck on a local minimum.

[1]This figure is inspired by a similar figure in Nervana's Demystifying Deep Reinforcement Learning blog post.

Chapter 3

Experiment

3.1 Playing A Game Of 'Catch'

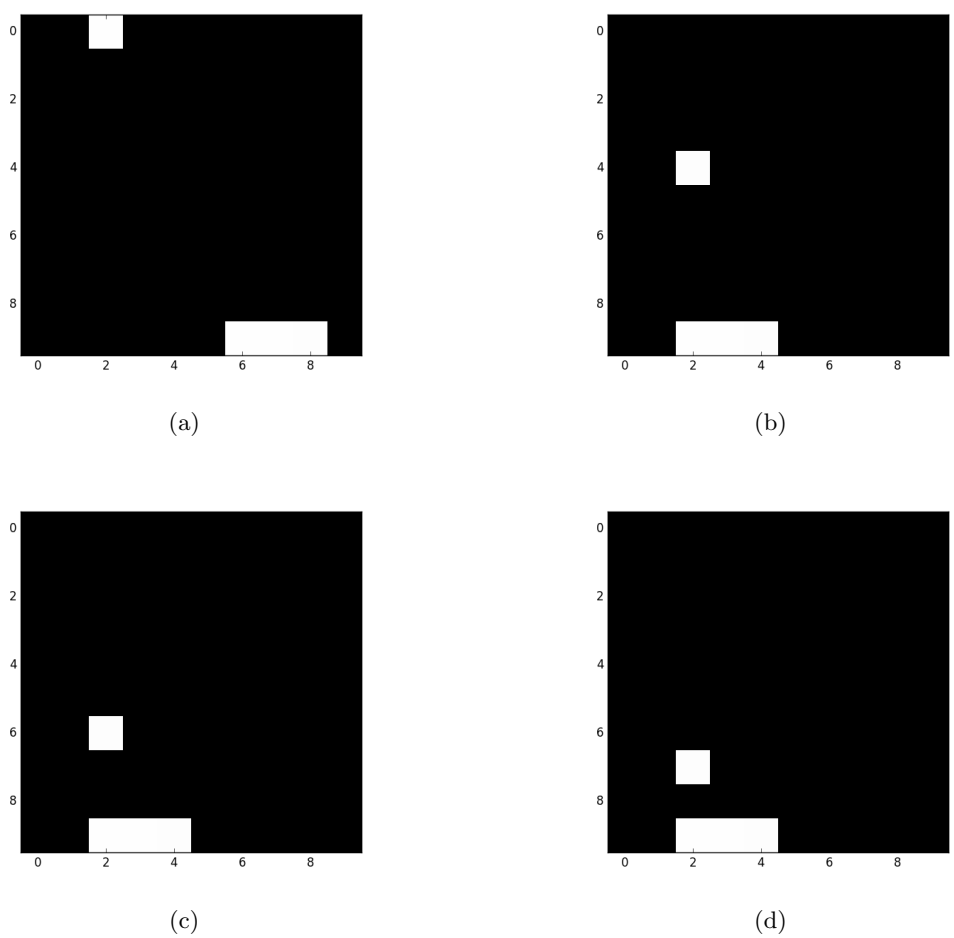


Figure 3.1: Using *keras* deep learning library to play a game of 'catch'

The following experiment is based on a reinforcement learning example described in the following blog post. This example uses the *keras* library [Chollet, 2015] and tensorflow

within python to build a deep Q-network as described in section 2.2 which uses reinforcement learning to learn how to play a game of catch. The game consists of a 10x10 pixel game screen as seen in figure 3.1, a 3 pixel 'basket' which moves around attempting to catch a 1 pixel 'fruit'. Intuitively, the fruit falls only one pixel per time step. The rewards for the agent are +1 if it catches the fruit and -1 if it misses it. The input for the network is the 10x10 game screen grid and the outcomes are the 3 possible actions (move left, move right, don't move). The code below shows the convolutional neural network used for this example:

Listing 3.1: Convolutional Neural Network Code

```
# parameters
   num_actions = 3
   hidden_size = 100
   grid_size = 10

   model = Sequential()
   model.add(Dense(hidden_size, input_shape=(grid_size**2,), activation='relu'))
   model.add(Dense(hidden_size, activation='relu'))
   model.add(Dense(num_actions))
   model.compile(sgd(lr=.2), "mse")
```

where the input layer is a convolutional layer taking a whole 10x10 grid as input, the two hidden layers consist of 100 neurons and the mean squared error is used to measure loss. Unlike the Deepming Neural Network model specified in table 2.1 which is used to play Atari games, our game of 'catch' is far less complex and does not seem to require the same level of model complexity. This is due to the fact that our input game screen grid consists of 100 pixels in a 10x10 input layer whilst the Deepmind algorithms take in four 84x84 game screens through the input layer. Also our agent can only perform 3 actions meaning there can only be 3 outputs from the final layer compared to the 18 outputs (potential actions) from the Atari games. The algorithm used in this example uses a ϵ-greedy policy to take care of the exploration-exploitation problem as shown below:

Listing 3.2: E-Greedy Policy Code

```
epsilon = .1
while not game_over:
          input_tm1 = input_t
          # get next action
          if np.random.rand() <= epsilon:
              action = np.random.randint(0, num_actions, size=1)
          else:
              q = model.predict(input_tm1)
              action = np.argmax(q[0])
```

where $input_tm1$ is the initial input from observing the current environment, q uses the model specified in listing 3.1 to make Q-value predictions on the current state and *action* is the greedy action for the current state. The value of epsilon used originally is $\epsilon = 0.1$. The original code from this blog post can be found here.

3.1.1 Performance Diagnostics

Running the code

We use the parameters specified above to train our reinforcement learning model. We first need to save the training code from the blog post as *qlearn.py* and the testing code as *qtest.py*. We initially run the training code using the following command:

```
$ python qlearn.py
```

This will save the model weights and architecture in our working directory as files named *model.h5* and *model.json*. The *qtest.py* uses these weights and model architecture from the model training in order to test the agents performance. We use the following command to do so:

```
$ python qtest.py
```

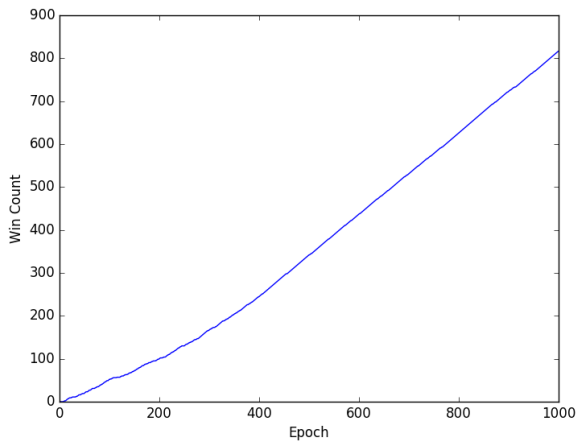
The testing code generates figures in the working directory as shown in figure 3.1, which can be used to visually prove that the agent is indeed learning how to play the game. A gif can be generated from these figures, however this requires the installation of *ffmpeg* package beforehand. The following commands will install ffmpeg and generate a gif file showing the agent playing the game:

```
$ sudo apt-get install ffmpeg
$ ffmpeg -i %03d.png output.gif -vf fps=1
```

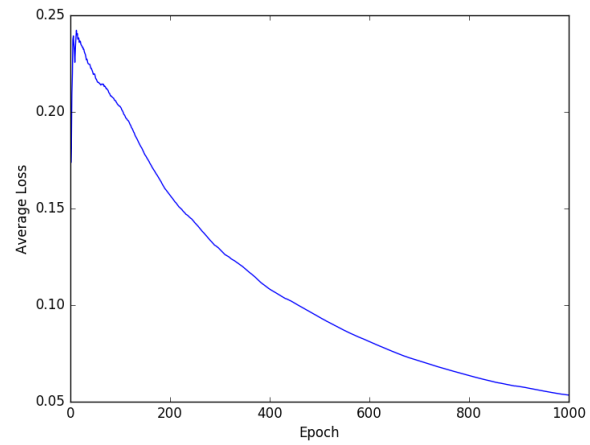
This gif gives a visual representation of a 3 pixel agent in a 10x10 grid game screen with a 1 pixel 'fruit' falling one pixel at each time point. The testing agent seems to perform well and the gif shows the agent catching all the falling 'fruits' [1].

We will use the code given in the appendix to measure and perform certain performance diagnostics of interest. The quantities which we are interested in are the agent's win count and reward and the model loss. In figure 3.2 we can see the win count of the agent during model training over the 1000 training epochs and the average model loss over the same time. It is evident from the graph that the agent seems to be learning how to play the game towards later time steps. The win count line graph becomes steeper and resembles a $y = x$ graph as after approximately the 400th epoch. The average training loss is also decreasing (almost exponentially) with time due to the parameter optimisation within the neural network. These two graphs show satisfactory trends in model training. In figure 3.3 we can see the average Q-value of the chosen action and the largest Q-value per epoch throughout the 1000 time steps. We note that the average Q-value of the chosen action increases to around 0.6 whilst the largest Q-value at each epoch seems to 'converge' to 1.0 as the agent learns how to play the game. It is interesting to note in the average Q-value graph that around the 100th epoch there is a dip in the graph. This is due to the ϵ-greedy policy which we have introduced. This has led to the agent choosing non-greedy actions in order to explore.

[1] The generated gif is the same as the one on top of the Keras blog post where this experiment is inspired from.



(a) Training Win Count



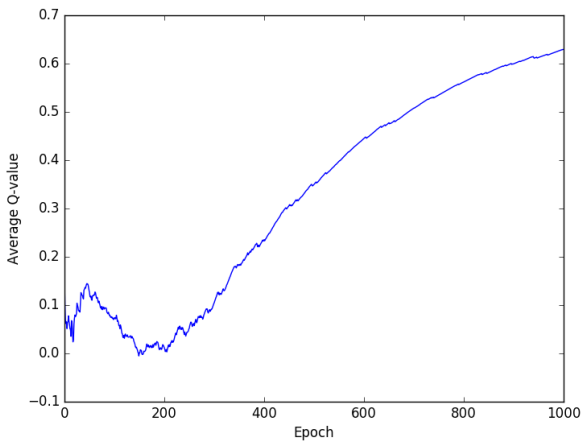
(b) Average Training Loss

Figure 3.2: Model Training Loss And Win Count

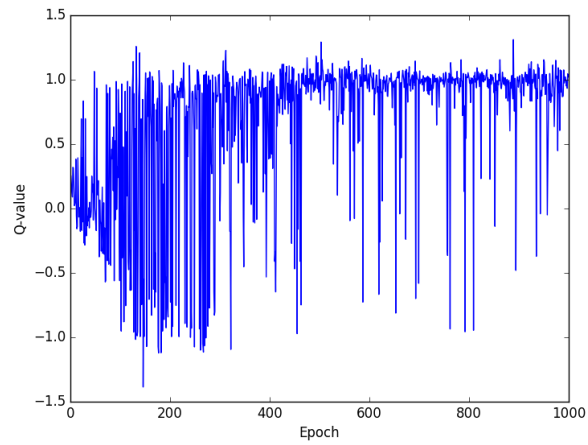
We would ultimately like the model we have built during training to perform well in testing. During testing, we use the model parameters saved during model training in order to test the performance of our agent in a real game situation. After running our model testing code, we have measured the reward count of the agent during training and testing and plotted them in figure 3.4. In the training plot we note a initial dip of the reward count to negative values. This is due to the fact that the agent is given a -1 reward if it does not catch the 'fruit'. The reward count picks up to positive values around the 200th epoch and from that point onwards the agent seems to be performing well by losing very few games. Model testing seems to also be successful with the agent winning all 100 games as seem on the right hand plot of figure 3.4.

3.1.2 Diminishing Epsilon Performance

Although our model seems to perform well during model testing, we would like to improve the performance of our agent whilst training. It seems counter intuitive to have the same epsilon value throughout the 1000 training epochs. The agent is supposed to learn how to play the game and whilst environment exploration is important in the beginning, it can be argued that a 10% chance of choosing the 'wrong' path whilst the agent seems to have picked up the optimal route to win is too high. In figure 3.4 we note that after the 400th training epoch, the agent seems to be an expert at the game. The game which the agent is playing is not very complex and we believe it is not smart to continue exploring after in the latter epochs. Throughout the next 600 epochs, the agent will choose the non-greedy (and probably incorrect) actions 60 times. We note various small 'dips' of the line in this graph coming from negative rewards due to path exploration. We have modified the ϵ-greedy algorithm shown originally in listing 3.2 so the value of epsilon decreases as the agent learns how to play the game. The following code shows how we have achieved this:



(a) Average Q-value of chosen actions



(b) Largest Q-value per epoch

Figure 3.3: Q-value Diagnostics

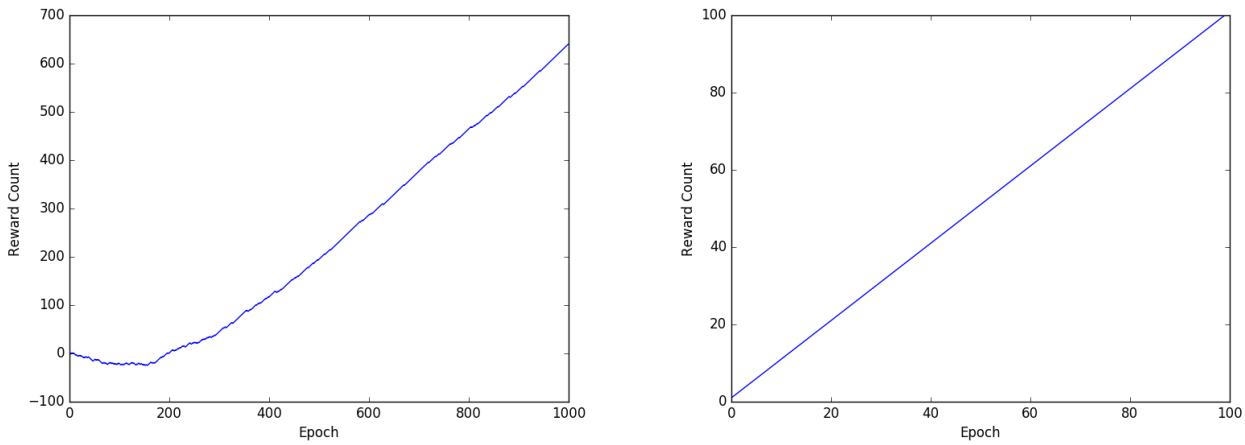
Listing 3.3: Diminishing epsilon code Code

```
# get next action
if np.random.rand() <= epsilon*0.99**e:
# e is the epoch order
```

In this listing the value of e is the order of the current epoch. The value of epsilon is initially 0.1 and it decreases as the epoch order increases. This leads to the agent choosing greedy actions more often as the training epochs progress. In figure 3.5 we note a clear improvement in agent performance using the modified diminishing epsilon code from listing 3.3. The other values of epsilon in this plot are 0 (completely greedy), 0.01 and 0.1. We note that the fully greedy agent performs better at first, however due to not exploring, it falls short eventually in terms of total reward due to the fact that it probably has not figured out the optimal route to winning the game and maximising the score. The diminishing epsilon agent and the agent with a constant ϵ=0.1 have a very similar start with a big negative reward dip. However due to learning the game at approximately the same rate initially and then choosing the greedy actions, the agent with a diminishing epsilon value obtains the largest total reward (almost 100 reward points more than the agent with a constant epsilon) by avoiding unnecessary environment exploration. The total reward line for this agent in the later epochs resembles a perfectly straight linear model with one parameter and gradient 1.

3.1.3 Varying Layer Size Performance

Another interesting modification of our model is the choice of the training model architecture. It is usually hard to define a model that will usually perform better than any other model without some initial trial and error experimentation. Generally, you need a network large enough to capture the structure of the problem if that helps at all. As previously noted, the Deepmind model from table 2.1 seems to be excessive for our game. The model



(a) Training Reward Count

(b) Testing Reward Count

Figure 3.4: Training vs Testing Accumulated Reward

chosen from the creators and writers of this code is the one mentioned in listing 3.1. We would like to check if increasing model complexity really improves training performance and if so, is it too computationally expensive to be considered? From table 3.1 we observe the time taken for the 1000 epochs of training to be completed with different hidden layer sizes. We observe an approximately 30 second increase in training time from 50 neurons to 100 and then another 30 seconds from 100 to 150. We run the following models and measure the total reward received from the agent [2]. This is shown in figure 3.6 where we note that the best performing model is the original model with hidden layers of size 100. The less complex models (30 and 50 neurons) do not seem to be as satisfactory as the 100 neuron model. Even though they take less time to perform the 1000 training epochs, one could argue that the model with 100 neurons does not need as many training epochs to learn the optimal winning playing method.

	30 Neurons	50 Neurons	100 Neurons	150 Neurons
Time Taken	497.32	509.56	534.41	563.13

Table 3.1: Time taken to train the model for different hidden layer sizes

[2]Note that the diminishing epsilon ϵ-greedy algorithm is used for all models.

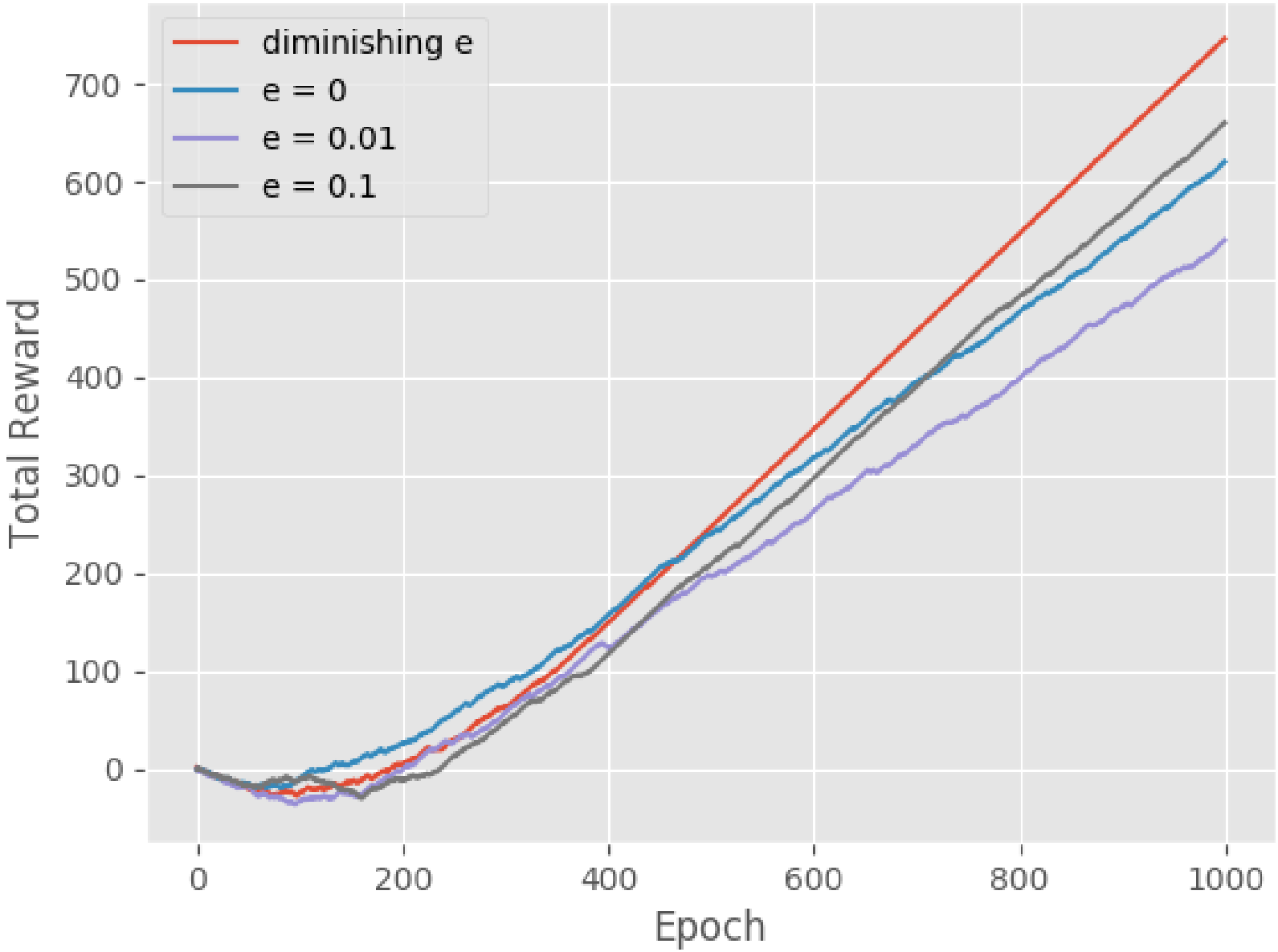


Figure 3.5: Training set performance for different values of ϵ

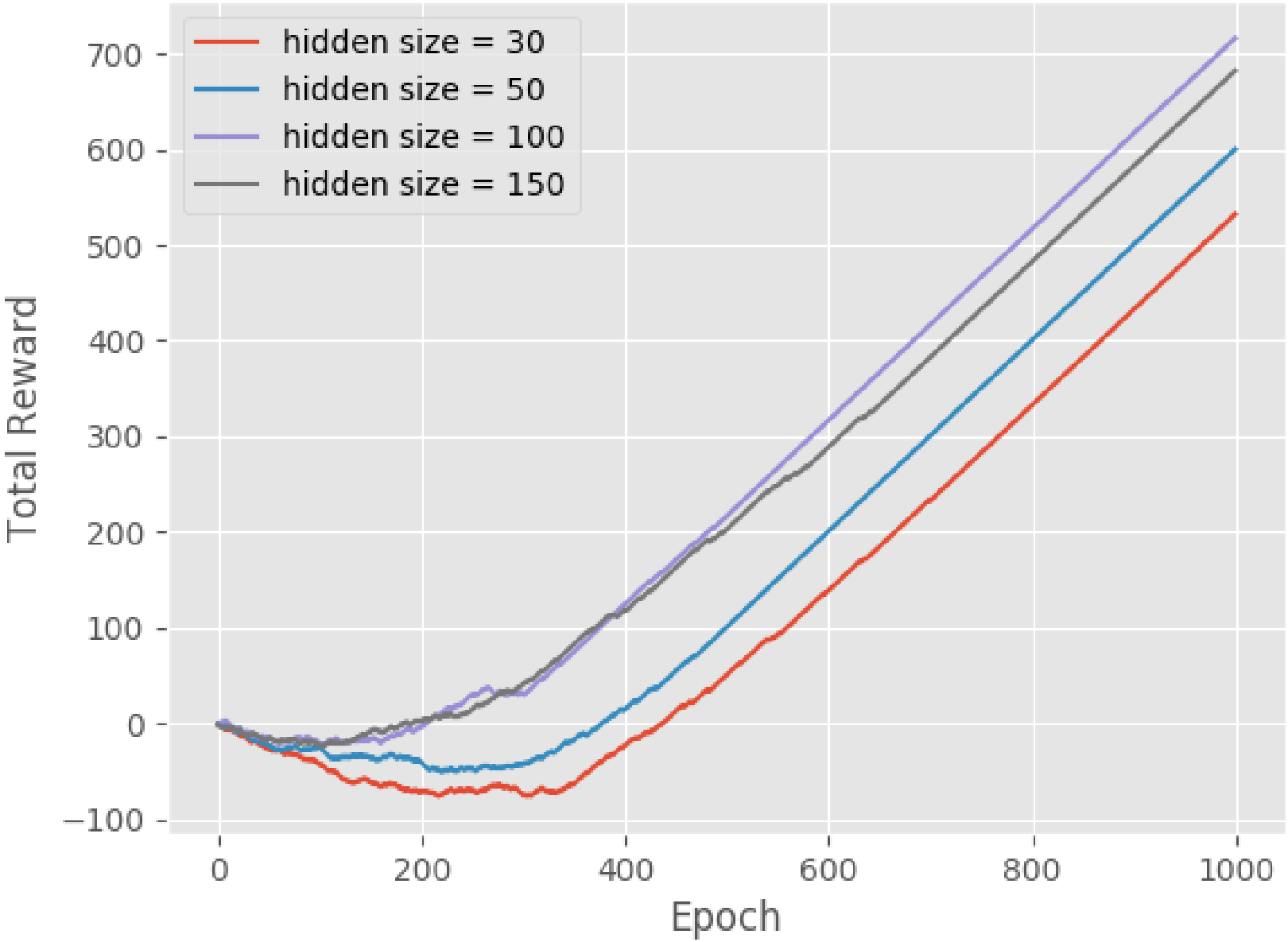


Figure 3.6: Training performance for different hidden layer sizes

Chapter 4

Conclusions And Further Work

We have presented in this paper an introduction to Deep Reinforcement Learning and some important concepts and identities which are essential in reinforcement learning algorithms. Let us summarise the reinforcement learning elements we have presented in this topic. Chapter one of this project gives an introduction to the importance of Bayesian methods and Markov Decision Processes in reinforcement learning problems where environment interaction is essential. The reinforcement learning agent and its environment interact over a sequence of discrete time steps. The agent interacts with the environment by performing a pre-defined task where it chooses actions at different states of the environment. After each action taken the agent is given a reward which it then uses to evaluate the choice of actions in the future. The agent is given an objective function which it tries to maximise.

An important concept introduced in this chapter is the exploration-exploitation trade-off problem which is typically mitigated by ϵ-greedy policies. The ϵ-greedy methods choose random actions a small fraction of the time, and the softmax methods grade their action probabilities according to the current action-value estimates. An interesting policy which we addressed in this paper was to use a non-constant value of ϵ throughout training. Since the agent seems to learn how to play the game fairly quickly, we decided that exploration should not have the same probability for latter training epochs. We adapted the training code in order to decrease the value of ϵ as the training progresses. This led to a higher accumulated reward for training as shown in figure 3.5.

These ϵ-greedy algorithms were used throughout this project, however there are other policies we could consider further. One promising idea mentioned by [Sutton and Barto, 1998] is to use estimates of the uncertainty of the action-value estimates to direct and encourage exploration. Consider two non-greedy actions to have estimated values slightly less than the greedy action. These actions however differ in uncertainty. The estimated value for one action is nearly certain due to the action being previously chosen many times, whilst the other action is well known and has a high uncertainty in its value prediction. The true value of the second action could be greater than the greedy action, hence it makes more sense to explore this action compared to the one with a low uncertainty.

In section 3.1.3 we discussed the difficulties which arise when building deep neural networks. We decided that building a network as complex as the one used by Deepmind was not necessary for our simple game. Hence we used a two-layer network with a convolutional

input layer. The difficulties in this case arise when choosing the layer sizes for the hidden layers where some trial and error exploration is necessary. In figure 3.6 we can see that the model with 100 neurons in the hidden layer performs better than others. However we usually need to address the question whether an increase in model complexity is worth the increase in computational time. In this project we have used the same number of neurons in each hidden layer which could not be the best performing network possible.

Multi-agent environments are interesting extensions to the Deep Q-Learning model we have used in this chapter. Different agents are controlled by autonomous Deep Q-Networks and interact when sharing a complex environment. Multi-agent systems appear naturally in social, economic or political scenarios. An interesting paper which observes the effects of two agents playing a game of Pong whilst using independent Deep Q-networks is [Tampuu et al., 2015]. This paper uses game theory principles to assign rewards and objective functions to each agent. I am currently using these principles to build a multi-agent based model of the economy which takes into account different population needs based on a geographical scale [1].

[1] The incomplete code can be found here

Chapter 5

Appendix

As mentioned above, the original code from this blog post can be found here. The following is the same code with a few amendments. We have used the same testing code as the one from the blog post.

Training

Listing 5.1: Modified Training Loop Code

```
import json
import numpy as np
from keras.models import Sequential
from keras.layers.core import Dense
from keras.optimizers import sgd
import matplotlib.pyplot as plt
import time

class Catch(object):
    def __init__(self, grid_size=10):
        self.grid_size = grid_size
        self.reset()

    def _update_state(self, action):
        """
        Input: action and states
        Ouput: new states and reward
        """
        state = self.state
        if action == 0: # left
            action = -1
        elif action == 1: # stay
            action = 0
        else:
            action = 1 # right
        f0, f1, basket = state[0]
        new_basket = min(max(1, basket + action), self.grid_size-1)
```

```
        f0 += 1
        out = np.asarray([f0, f1, new_basket])
        out = out[np.newaxis]

        assert len(out.shape) == 2
        self.state = out

    def _draw_state(self):
        im_size = (self.grid_size,)*2
        state = self.state[0]
        canvas = np.zeros(im_size)
        canvas[state[0], state[1]] = 1 # draw fruit
        canvas[-1, state[2]-1:state[2] + 2] = 1 # draw basket
        return canvas

    def _get_reward(self):
        fruit_row, fruit_col, basket = self.state[0]
        if fruit_row == self.grid_size-1:
            if abs(fruit_col - basket) <= 1:
                return 1
            else:
                return -1
        else:
            return 0

    def _is_over(self):
        if self.state[0, 0] == self.grid_size-1:
            return True
        else:
            return False

    def observe(self):
        canvas = self._draw_state()
        return canvas.reshape((1, -1))

    def act(self, action):
        self._update_state(action)
        reward = self._get_reward()
        game_over = self._is_over()
        return self.observe(), reward, game_over

    def reset(self):
        n = np.random.randint(0, self.grid_size-1, size=1)
        m = np.random.randint(1, self.grid_size-2, size=1)
        self.state = np.asarray([0, n, m])[np.newaxis]


class ExperienceReplay(object):
    def __init__(self, max_memory=100, discount=.9):
        self.max_memory = max_memory
        self.memory = list()
```

```
        self.discount = discount

    def remember(self, states, game_over):
        # memory[i] = [[state_t, action_t, reward_t, state_t+1], game_over?]
        self.memory.append([states, game_over])
        if len(self.memory) > self.max_memory:
            del self.memory[0]

    def get_batch(self, model, batch_size=10):
        len_memory = len(self.memory)
        num_actions = model.output_shape[-1]
        env_dim = self.memory[0][0][0].shape[1]
        inputs = np.zeros((min(len_memory, batch_size), env_dim))
        targets = np.zeros((inputs.shape[0], num_actions))
        for i, idx in enumerate(np.random.randint(0, len_memory,
                                                  size=inputs.shape[0])):
            state_t, action_t, reward_t, state_tp1 = self.memory[idx][0]
            game_over = self.memory[idx][1]

            inputs[i:i+1] = state_t
            # There should be no target values for actions not taken.
            # Thou shalt not correct actions not taken #deep
            targets[i] = model.predict(state_t)[0]
            Q_sa = np.max(model.predict(state_tp1)[0])
            if game_over: # if game_over is True
                targets[i, action_t] = reward_t
            else:
                # reward_t + gamma * max_a' Q(s', a')
                targets[i, action_t] = reward_t + self.discount * Q_sa
        return inputs, targets

if __name__ == "__main__":
    # parameters
    epsilon = 0.1 # exploration
    num_actions = 3 # [move_left, stay, move_right]
    epoch = 1000
    max_memory = 500
    hidden_size = 100 # change to 50 and save a[] as hidden_50.csv
    batch_size = 50
    grid_size = 10
    print(hidden_size)

    model = Sequential()
    model.add(Dense(hidden_size, input_shape=(grid_size**2,), activation='relu'))
    model.add(Dense(hidden_size, activation='relu'))
    model.add(Dense(num_actions))
    model.compile(sgd(lr=.2), "mse")

    # If you want to continue training from a previous model, just uncomment the
        line bellow
    # model.load_weights("model.h5")
```

```python
# Define environment/game
env = Catch(grid_size)

# Initialize experience replay object
exp_replay = ExperienceReplay(max_memory=max_memory)

# Train
win_cnt = 0
pp = []
ww = []
w_av = []
p_av = []
start = time.time()
for e in range(epoch):
    loss = 0.
    env.reset()
    game_over = False
    # get initial input
    input_t = env.observe()
    #while (not game_over and win_cnt<=499):
    #while win_cnt <= 200:
    while not game_over:
        input_tm1 = input_t
        # get next action
        if np.random.rand() <= epsilon*0.99**e:
            action = np.random.randint(0, num_actions, size=1)
        else:
            q = model.predict(input_tm1)
            action = np.argmax(q[0])

        # apply action, get rewards and new state
        input_t, reward, game_over = env.act(action)
        if reward == 1:
            win_cnt += 1

        # store experience
        exp_replay.remember([input_tm1, action, reward, input_t], game_over)

        # adapt model
        inputs, targets = exp_replay.get_batch(model, batch_size=batch_size)

        loss += model.train_on_batch(inputs, targets)

    pp.append(reward)
    p_av.append(sum(pp))
    print("Epoch {:03d}/999 | Loss {:.4f} | Win count {}".format(e, loss,
        win_cnt))
end = time.time()
time_taken = end - start
print("Time taken {}".format(time_taken))
```

```
    # Save trained model weights and architecture, this will be used by the
        visualization code
    model.save_weights("model.h5", overwrite=True)
    with open("model.json", "w") as outfile:
        json.dump(model.to_json(), outfile)
        ep = np.r_[1:len(p_av)+1]
        a = np.asarray(p_av)
        a.tofile('hidden_100.csv',sep=',',format='%10.5f')
        plt.plot(p_av)
        plt.xlabel('Epoch')
        plt.ylabel('Q-value')
        plt.yscale('linear')
        plt.savefig(".png")
```

Plots

Listing 5.2: Line plot code

```
import matplotlib.pyplot as plt
import ggplot
from ggplot import *
import numpy as np
import os

# -------------------- Multi-line Plot ---------------------------- #

hidden_50 = np.genfromtxt('hidden_50.csv', delimiter=',')
hidden_100 = np.genfromtxt('hidden_100.csv', delimiter=',')
hidden_150 = np.genfromtxt('hidden_150.csv', delimiter=',')
hidden_30 = np.genfromtxt('hidden_30.csv', delimiter=',')
x = np.r_[1:len(hidden_30)+1]

data_matrix = np.column_stack((x, hidden_30, hidden_50, hidden_100, hidden_150))
df3 = DataFrame(data_matrix, columns = ['epoch', 'hidden_30', 'hidden_50',
    'hidden_100', 'hidden_150'])


plt.style.use('ggplot')
plt.plot(hidden_30)
plt.plot(hidden_50)
plt.plot(hidden_100)
plt.plot(hidden_150)

plt.legend(['hidden size = 30', 'hidden size = 50', 'hidden size = 100', 'hidden
    size = 150'], loc='upper left')
plt.xlabel('Epoch')
plt.ylabel('Total Reward')
plt.savefig("hidden.png")
```

```
# -------------------- Testing Plot -------------------------- #

env = Catch(grid_size)
    c = 0
    pp, ww, w_av, p_av = [], [], [], []
    for e in range(100):
        loss = 0.
        env.reset()
        game_over = False
        # get initial input
        input_t = env.observe()

        #plt.imshow(input_t.reshape((grid_size,)*2),
         #         interpolation='none', cmap='gray')
        #plt.savefig("%03d.png" % c)
        c += 1
        while not game_over:
            input_tm1 = input_t

            # get next action
            q = model.predict(input_tm1)
            action = np.argmax(q[0])

            # apply action, get rewards and new state
            input_t, reward, game_over = env.act(action)
        print(max(q[0]))
        print(q)
        pp.append(max(q[0]))
        p_av.append(np.sum(pp))
    plt.plot(pp)
    plt.xlabel('Epoch')
    plt.ylabel('Q-value')
    plt.yscale('linear')
    plt.savefig("qvalue-test.png")

    plt.plot(p_av)
    plt.xlabel('Epoch')
    plt.ylabel('Average Q-value')
    plt.yscale('linear')
    plt.savefig("qvalue_av_test.png")
```

Bibliography

Dimitri P Bertsekas. J.. n. tsitsiklis. *Neuro-dynamic Programming*, 1996.

Sébastien Bubeck, Nicolo Cesa-Bianchi, et al. Regret analysis of stochastic and nonstochastic multi-armed bandit problems. *Foundations and Trends® in Machine Learning*, 5(1): 1–122, 2012.

François Chollet. Keras. https://github.com/fchollet/keras, 2015.

Evgeniĭ Dynkin. Markov processes.

Robert Hecht-Nielsen et al. Theory of the backpropagation neural network. *Neural Networks*, 1(Supplement-1):445–448, 1988.

Geoffrey E Hinton and Ruslan R Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.

Long-Ji Lin. *Reinforcement learning for robots using neural networks*. PhD thesis, Fujitsu Laboratories Ltd, 1993.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

Anna W Roe, Sarah L Pallas, Young H Kwon, and Mriganka Sur. Visual projections routed to the auditory pathway in ferrets: receptive fields of visual neurons in primary auditory cortex. *Journal of Neuroscience*, 12(9):3651–3664, 1992.

Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

Ardi Tampuu, Tambet Matiisen, Dorian Kodelja, Ilya Kuzovkin, Kristjan Korjus, Juhan Aru, Jaan Aru, and Raul Vicente. Multiagent cooperation and competition with deep reinforcement learning. *arXiv preprint arXiv:1511.08779*, 2015.

Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3-4):279–292, 1992.